

Teaching Verification-Centered Software Engineering with Coding Agents

From candidate patches to evidence-backed claims and human judgment

Chenxi Luo · Moon-Kuen Mak · Wei Xiang · Tiejian Luo*

La Trobe University · University of Chinese Academy of Sciences

NeurIPS 2026 Education Track · * Corresponding author: tjluo@ucas.ac.cn

What this module is—and is not

Self-improving agent course	This module
Improve the agent	Improve the engineering decision
Train/RL, search, reward models	Validate intent and bound the agent
Planner or agent architecture	Strengthen independent evidence
Benchmark/task performance	Calibrated claim + justified approval
Original agent research	Reusable 150-minute SE workshop

The agent may improve its candidate; the learner must improve the evidence and judgment.

Why now: longer agent tasks, larger review burden

As agents act across more files, tools, and steps, a final green signal hides a longer chain of assumptions and actions.

- ▶ More search can improve a candidate.
- ▶ More autonomy can expand the impact of a mistake.
- ▶ Neither defines the evidence required to accept a software change.

The educational problem is trustworthy engineering judgment around agent-produced change.

Learning outcomes

By the end, you can:

- ▶ classify correctness evidence, revision guidance, model opinion, and approval;
- ▶ execute an evidence-to-approval lifecycle under a bounded ACI;
- ▶ expose a weak verifier with boundaries and mutations;
- ▶ write a property-scoped claim with residual uncertainty;
- ▶ transfer architecture/security judgment to authorization.

The emerging technical concept: improve the engineer

Coding agents no longer emit code only once. They:

1. inspect a repository;
2. propose a candidate change;
3. execute tools and receive feedback;
4. revise the candidate;
5. stop under an explicit condition.

Our learner—not the agent—is the primary object of improvement.

The success criterion is reproducible evidence, a correctly scoped claim, and a justified decision—not a higher agent benchmark score.

The full evidence-to-approval lifecycle

Intent I -> validate -> Specification S

|

v

Agent + ACI + Environment X

|

v

Candidate C

|

v

Verifier V -> Evidence E / Counterexample F -> Revision guidance R

^

|

|

|

|

|

|

v

Property-scoped claim K

|

v

A compact technical model

- ▶ *S*: intended behavior or specification
- ▶ *I*: stakeholder intent before validation
- ▶ *X*: repository and execution environment
- ▶ *C*: generated candidate
- ▶ *V*: executable verifier
- ▶ *E*: supporting evidence
- ▶ *F*: falsifying counterexample
- ▶ *R*: guidance about what to revise next
- ▶ *K*: property-scoped claim
- ▶ *J*: accountable engineering judgment

$I \rightarrow S \rightarrow (\text{Agent} + \text{ACI}, X) \rightarrow C \rightarrow V \rightarrow \{E, F\} \rightarrow R \rightarrow C' / K \rightarrow J$

Three signals that must not be collapsed

Signal	Question answered	Examples
Correctness evidence	What property is supported?	tests, mutation/static analysis, proof, policy check
Revision guidance	What should be tried next?	reward, self-evaluation, revision distance, search score
Approval judgment	Should this enter the lifecycle?	architecture, security, compliance, rollback, owner decision

Model explanation is a fourth category: an opinion or rationale, not independent evidence.

Signal-classification checkpoint

Classify each item before deciding what it permits:

Item	Category
pytest: 12 passed	correctness evidence, scoped to named tests/environment
reward-model score 0.91	revision guidance
LLM says “looks correct”	model opinion
static analyzer: no finding	correctness evidence, scoped to its rules
architect rejects global state	approval judgment with architecture evidence
accountable reviewer approves	approval decision

No single row establishes universal correctness.

Deliberate scope boundary

This workshop does **not** teach:

- ▶ model training or reinforcement learning;
- ▶ inference-time scaling or repeated-sampling algorithms;
- ▶ MCTS, planner optimization, or agent architecture search;
- ▶ prompt optimization, memory systems, or meta-agents.

Those techniques may improve an agent. Here, the agent is held fixed or bounded so learners can improve evidence, claim calibration, and engineering judgment.

What makes a verifier independent?

Independent evidence does not merely repeat the candidate's assumptions.

Weak signal

Agent says “looks correct”
Test duplicates implementation
Same model grades itself
“All tests passed”

Stronger signal

Test with a separately designed oracle
Boundary or property from validated intent
Tool, checker, or accountable reviewer
Named tests, environment, logs, and limitations

Three limits of executable verification

1. **Specification risk:** S may not represent real intent.
2. **Verifier risk:** V may omit a behavior or accept a mutation.
3. **System risk:** passing behavior may violate architecture or security.

Verification reduces uncertainty; it does not create universal correctness.

Four lifecycle states

generated -> candidate -> property-scoped verified -> approved

- ▶ Generated: an artifact exists.
- ▶ Candidate: it is concrete and reproducible enough to evaluate.
- ▶ Verified: named evidence supports named properties under a named environment.
- ▶ Approved: an accountable process accepts the next lifecycle action.

Four research surfaces—and our fifth layer

Work	Verifier/feedback role
EvalPlus, NeurIPS 2023	weak evidence: augmented tests and mutations
SWE-agent, NeurIPS 2024	constrained interaction: API actions and feedback
CLEVER, NeurIPS 2025	specification fidelity: semantic equivalence + proof
ReLoc, NeurIPS 2025	revision guidance: candidate evaluation + local search
This module	claim calibration + engineering approval

Workshop route

Validate intent

- > classify signals
- > inspect candidate provenance
- > run weak verifier
- > design falsification
- > revise and re-run
- > inspect architecture
- > scope claim
- > human decision
- > transfer

Lab 1: specification risk comes first

Prompt: “Make discounts reliable near the threshold.”

Before code:

- ▶ separate observed, inferred, and validated claims;
- ▶ identify the user, scenario, and desired value;
- ▶ ask whether the threshold is inclusive;
- ▶ design acceptance examples;
- ▶ record how the requirement was validated.

Lab 2: candidate provenance

Candidate C-017 includes:

- ▶ a frozen baseline;
- ▶ an exact unified diff;
- ▶ a provenance manifest;
- ▶ learner-visible tests;
- ▶ state candidate, not verified.

A live coding agent is optional. The prepared candidate makes the result reproducible without an API key.

Lab 3: the weak verifier passes

```
$ pytest tests/test_visible.py -q  
4 passed
```

Question: what plausible incorrect implementation would still pass?

Do not inspect the instructor suite until you have designed and executed your own falsification cases.

Facilitator pause: learner falsification attempt

Stop here before revealing the worked counterexample.

Learners should now:

1. commit at least one plausible mutation;
2. design tests that would kill it;
3. execute those tests against Candidate C-017;
4. record evidence and residual uncertainty.

Continue only after artifacts have been collected.

Strengthen the oracle

Use:

- ▶ equivalence classes;
- ▶ exact boundaries and just-below/above cases;
- ▶ invalid inputs;
- ▶ plausible mutations;
- ▶ properties that do not copy implementation structure.

The stronger suite reveals a counterexample at the exact threshold.

Lab 4: same outputs, different systems

Both candidates pass 12 shared functional tests.

Investigate:

- ▶ process-global mutable state;
- ▶ direct environment access;
- ▶ dependency direction;
- ▶ duplicated validation;
- ▶ behavior after configuration changes;
- ▶ test isolation and rollback consequences.

An executable architecture counterexample

Candidate A, default policy: 12.0

Candidate A, after rate changes: 12.0 <- stale cache

Candidate B, explicit new policy: 24.0

Functional examples did not encode determinism under configuration change.

Bound the agent, not only the prompt

A task contract names:

- ▶ objective and validated acceptance criteria;
- ▶ included and excluded paths;
- ▶ allowed tools and forbidden actions;
- ▶ verification commands and expected results;
- ▶ stop and escalation conditions;
- ▶ actions requiring human approval.

For the same task, compare an unrestricted shell contract with the bounded contract: identify which actions, observations, guardrails, and escalation conditions change. This is interface-design evidence, not a claim that the underlying model improved.

Lab 5: non-isomorphic transfer

New domain: document authorization after collaborator offboarding.

Learners must rebuild:

- ▶ principals, resources, grants, revocations, and precedence;
- ▶ stale-state and share-link threat models;
- ▶ executable policy evidence;
- ▶ architecture, privacy, audit, and rollback judgment.

Copying the discount boundary test does not demonstrate transfer.

Assessment and pilot plan

Measure:

- ▶ calibrated pre/post concept answers;
- ▶ correct separation of evidence, guidance, opinion, and approval;
- ▶ proportion of claims linked to executable evidence;
- ▶ hidden insufficiency discovered before release;
- ▶ architecture decision quality;
- ▶ non-isomorphic transfer score;
- ▶ completion time and learner friction.

Do not grade prompt count, token count, or lines generated.

Takeaway

The agent proposes. Evidence scopes the claim. Engineering judgment approves or rejects.

Our learner—not the agent—is the primary object of improvement.

Resources include slides, notebook, runnable repositories, rubrics, pilot protocol, and reproducible build.

References: EvalPlus (2023); SWE-agent (2024); CLEVER (2025); ReLoc (2025).